By Abhishek Dwivedi





Pizza Sales Analysis

Business Insights with MySQL





By Abhishek Dwivedi

Introduction

Hello! I'm Abhishek Dwivedi, an aspiring Data Analyst and currently working as a Data Analyst Intern.
I am passionate about working with data, uncovering meaningful insights, and solving real-world business problems through data analysis.
I have been developing my analytical and technical skills in SQL, Python, Power BI, Excel, and MySQL. I enjoy exploring datasets, writing SQL queries, identifying trends, and transforming raw data into meaningful information that can support better business decisions.
This Pizza Sales Analysis project is part of my learning journey, where I applied SQL concepts to analyze sales performance, revenue trends, customer ordering patterns, and product performance.
My Goal: To continuously improve my analytical skills and grow as a professional Data Analyst by working on real-world projects and solving data-driven business problems.

By Abhishek Dwivedi

WHAT WE DO

In this project, we analyze pizza sales data using MySQL to understand business performance and uncover meaningful insights.

Our analysis focuses on:

- Sales Analysis: Calculate total orders and overall revenue generated from pizza sales.
- Product Performance: Identify the highest-priced, most popular, and top-revenue-generating pizzas.
- Category Analysis: Evaluate pizza sales and revenue across different categories.
- Customer Ordering Trends: Analyze order distribution by hour and identify daily ordering patterns.
- Revenue Analysis: Calculate category-wise revenue contribution and track cumulative revenue over time.
- Business Insights: Use SQL queries to transform raw sales data into meaningful information that supports better business understanding and decision-making.
- Our Objective: To demonstrate how SQL can be used to analyze real-world sales data and solve practical business problems.

By Abhishek Dwivedi

1. Retrieve the total number of orders placed.

```sql
SELECT
    COUNT(order_id) AS total_orders
FROM
    ORDERS;
```

Result Grid

	total_orders
▶	21350

2. Calculate the total revenue generated from pizza sales.

```
SELECT
    ROUND(SUM(order_details.quantity * Pizzas.price),
            2) AS total_revenue
FROM
    order_details
        JOIN
    pizzas ON pizzas.pizza_id = order_details.pizza_id;
```

Result Grid

total_revenue
817860.05

By Abhishek Dwivedi

3. Identify the highest-priced pizza.

```
  SELECT
    pizza_types.name, Pizzas.price
FROM
    Pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY Pizzas.price DESC
LIMIT 1;
```

Result Grid

name	price
The Greek Pizza	35.95

4.Identify the most common pizza size ordered.

```
SELECT
    pizzas.size,
    SUM(quantity) AS most_common_pizza_size
FROM
    Pizzas
        JOIN
    order_details ON pizzas.Pizza_id = order_details.pizza_id
GROUP BY pizzas.size
ORDER BY most_common_pizza_size DESC ;
```

Result Grid | Filter Rows:

size	most_common_pizza_size
L	18956
M	15635
S	14403
XL	552
XXL	28

By Abhishek Dwivedi

5.List the top 5 most ordered pizza types along with their quantities.

```sql
SELECT
    pizza_types.NAME,
    SUM(order_details.quantity) AS most_ordered_pizza_types
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.NAME
ORDER BY most_ordered_pizza_types DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Expor

NAME	most_ordered_pizza_types
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

By Abhishek Dwivedi

6. Join the necessary tables to find the total quantity of each pizza category ordered.

```sql
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS total_order_quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.category
ORDER BY total_order_quantity DESC;
```

Result Grid | Filter Rows:

category	total_order_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

By Abhishek Dwivedi

7. Determine the distribution of orders by hour of the day.

```sql
SELECT
    HOUR(time) AS `HOUR`, COUNT(order_id) AS order_count
FROM
    orders
GROUP BY `HOUR`;
```

Result Grid

HOUR	order_cour
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642

By Abhishek Dwivedi

8. Count the number of pizza types in each category.

```
SELECT
    category, COUNT(`name`) `NAME`
FROM
    pizza_types
GROUP BY category;
```

Result Grid

	category	NAME
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9

By Abhishek Dwivedi

9. Group the orders by date and calculate the average number of pizzas ordered per day.

```sql
SELECT
    ROUND(AVG(avg_order), 0) AS avg_order_per_day
FROM
    (SELECT
        orders.`date`, SUM(order_details.quantity) AS avg_order
    FROM
        orders
    JOIN order_details ON orders.order_id = order_details.order_id
    GROUP BY orders.`date`) AS per_day;
```

Result Grid

avg_order_per_day
138

By Abhishek Dwivedi

10. Determine the top 3 most ordered pizza types based on revenue.

```
SELECT
    pizza_types.`name`,
    SUM(order_details.quantity * pizzas.price) AS revanue
FROM
    pizza_types
        JOIN
    pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.`name`
ORDER BY revanue DESC
LIMIT 3;
```

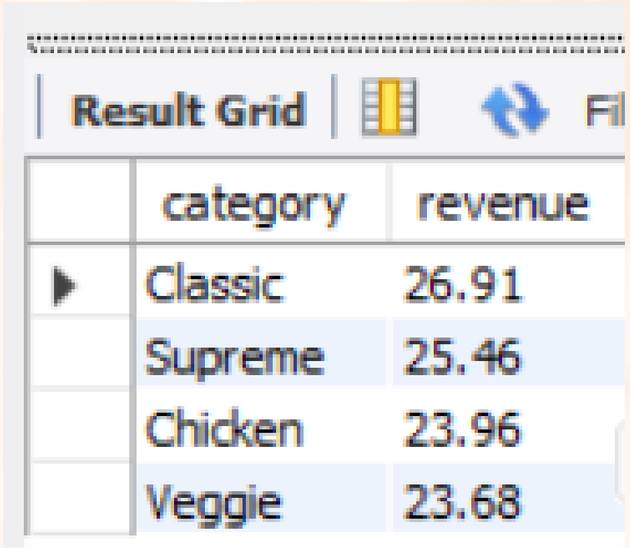
Result Grid | Filter Rows:

name	revanue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

By Abhishek Dwivedi

11. Calculate the percentage contribution of each pizza category to total revenue.

```
SELECT
    pizza_types.category,
    ROUND((SUM(order_details.quantity * pizzas.price) / (SELECT
                    ROUND(SUM(order_details.quantity * pizzas.price),
                                2) AS total_sales
                FROM
                    order_details
                        JOIN
                    pizzas ON pizzas.pizza_id = order_details.pizza_id)) * 100,
            2) AS revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue DESC;
```



Result Grid

	category	revenue
▶	Classic	26.91
	Supreme	25.46
	Chicken	23.96
	Veggie	23.68

By Abhishek Dwivedi

12. Analyze the cumulative revenue generated over time.

```
SELECT `date`,
SUM(revenue) OVER(ORDER BY `date`) AS cum_revenue
FROM
(SELECT orders.`date`,
ROUND(SUM(order_details.quantity*pizzas.price),0) AS revenue
FROM order_details
JOIN pizzas
ON order_details.pizza_id = pizzas.pizza_id
JOIN orders
ON orders.order_id = order_details.order_id
GROUP BY orders.`date`) AS sales ;
```

Result Grid | Filter Rows:

date	cum_revenue
2015-01-01	2714
2015-01-02	5446
2015-01-03	8108
2015-01-04	9863
2015-01-05	11929
2015-01-06	14358
2015-01-07	16560
2015-01-08	19398
2015-01-09	21525
2015-01-10	23989
2015-01-11	25861
2015-01-12	27780
2015-01-13	29830
2015-01-14	32357
2015-01-15	34342

By Abhishek Dwivedi

13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT `name`,ROUND(revenue,0) AS revenue_clear ,category FROM
(SELECT category, `name`,revenue    ,
RANK() OVER(PARTITION BY category ORDER BY revenue DESC ) AS `rank` FROM(SELECT
    pizza_types.category, pizza_types.`name`,
    SUM((order_details.quantity)*pizzas.price) AS revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY  pizza_types.category,pizza_types.`name`) AS Determine_top_3 ) Determine_top_3_2
WHERE `rank` <=3;
```

Result Grid | Filter Rows: | Export:

name	revenue_clear	category
The Thai Chicken Pizza	43434	Chicken
The Barbecue Chicken Pizza	42768	Chicken
The California Chicken Pizza	41410	Chicken
The Classic Deluxe Pizza	38180	Classic
The Hawaiian Pizza	32273	Classic
The Pepperoni Pizza	30162	Classic
The Spicy Italian Pizza	34831	Supreme
The Italian Supreme Pizza	33477	Supreme
The Sicilian Pizza	30940	Supreme
The Four Cheese Pizza	32266	Veggie
The Mexicana Pizza	26781	Veggie
The Five Cheese Pizza	26066	Veggie

CONCLUSION

Through this Pizza Sales Analysis project, I gained practical experience in using MySQL to analyze real-world sales data and solve business-related problems.

By working on 13 SQL business questions, I strengthened my understanding of data analysis techniques, including JOINs, Aggregate Functions, Subqueries, GROUP BY, and Window Functions.

This project helped me explore sales performance, revenue trends, popular pizza types, category-wise sales, and ordering patterns.

It also improved my ability to transform raw data into meaningful insights and strengthened my analytical and problem-solving skills.

Key Takeaway: Data is more than just numbers. With the right analytical approach and SQL skills, it can reveal valuable business insights and support better decision-making.

This project is another step forward in my journey toward becoming a professional Data Analyst.

Thank You for Exploring My Project!

LET'S CONNECT

Abhishek Dwivedi
Aspiring Data Analyst | Data Analyst Intern
Email: abhishek.dwivedi9945@gmail.com
GitHub: https://github.com/Abhishek07O
LinkedIn: https://www.linkedin.com/in/abhishek-dwivedi-5059b0301/
Instagram: https://www.instagram.com/abhishekdataanalyst/?hl=en

I'm eager to learn, collaborate, and connect with people passionate about data analytics.

Let's connect and grow together!